Membership Inference Attack in Code

Anonymous Author(s)
Affiliation
Address
email

Abstract

1

1 Introduction

Challenge I. Benchmark how to determine member and non-member.

Challenge II. code syntax.

Contribution. We summarize our contributions as follows:

2 Motivation

The inherent structural duality of source code – simultaneously embodying *human design intent* and *machine-enforced syntax* – introduces unique challenges for membership inference in code language models. Our analysis of XXX Python projects reveals that XXX% of code tokens are bound by syntactic constraints (e.g., `for-in`, `try-except`), forming rigid patterns that obscure the memorization signals crucial for membership detection. This syntactic determinism manifests two fundamental barriers:

- **Syntax-Induced Noise Amplification**: Language-mandated tokens like `in`, `except`, and `:` exhibit near-identical probability distributions across member and non-member samples. Their structural predictability drowns out discriminative signals from author-specific patterns.
- **Contextual Ambiguity**: Traditional token-level approaches cannot distinguish between *syntax-enforced occurrences* (e.g., `in` in `for x in data`) and *semantically meaningful instances* (e.g., `if key in dict`). This equivalence creates false attribution pathways.

The root challenge stems from code's *mandatory scaffolding* property: any functionally correct implementation must embed programmer intent within grammatical constraints. Whereas natural language allows infinite paraphrasing, code requires specific token sequences to satisfy the interpreter. This creates an adversarial scenario where the very tokens that define correctness (keywords, delimiters) act as persistent noise sources for membership inference.

Our approach leverages the *dual predictability hierarchy* inherent in code:

Language-Level Predictability: Syntax-enforced tokens follow deterministic patterns dictated by Python's grammar

Author-Level Predictability: Unique implementation choices (variable names, API chains) reflect training data memorization

By surgically separating these predictability layers through syntax-aware filtering, we amplify the discriminative power of membership signals while suppressing structural noise. This paradigm shift addresses the core limitation of existing methods – their inability to account for code's machine-enforced regularity – establishing a foundation for code-specific membership inference.

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

3 Methodology

3.1 Min-k% Prob with Syntax-Aware Filtering

We propose **Syntax-Aware Min-k% Prob**, an enhanced membership inference method specifically designed for code data. This extension addresses the fundamental difference between natural language and programming languages: code contains *syntax-enforced patterns* where specific token sequences are structurally determined (e.g., `for...in...`, `if...else...`). The core insight is that suffix tokens in these patterns (e.g., `in`, `else`) exhibit high prediction probabilities regardless of their membership status, introducing noise to the Min-k% Prob metric. Our method introduces a two-stage filtering process:

1. **Syntax Rule Construction**: Establish a comprehensive syntax rule set $\mathcal{R}$ based on Python's formal grammar specification. For each syntax-bound phrase $r_i \in \mathcal{R}$, we define:

$$r_i = (\phi_{\text{prefix}}, \phi_{\text{suffix}}, \tau)$$

   where:

   - ϕ_{prefix}: Context-sensitive prefix tokens (e.g., `for`, `try`)
   - ϕ_{suffix}: Structurally determined suffix tokens (e.g., `in`, `except`)
   - τ: AST node type for validation (e.g., `For`, `Try`)

   Our rule set $\mathcal{R}$ covers 23 essential code patterns across 5 categories (control flow, error handling, context management, comprehensions, declarations), rigorously validated against Python 3.11 documentation.

2. **Hybrid Pattern Detection**: For input code x, perform dual validation:
   (a) *Surface-Level Matching*: Scan tokens using regular expressions derived from ϕ_{prefix} and ϕ_{suffix}
   (b) *Semantic Validation*: Construct Abstract Syntax Tree (AST) to verify pattern validity through node type checking with τ

   This yields verified syntax phrases $\mathcal{P} = \{(p_1^{\text{pre}}, p_1^{\text{suf}}), ..., (p_n^{\text{pre}}, p_n^{\text{suf}})\}$.

3. **Context-Preserving Token Filtering**: For each detected $(p_i^{\text{pre}}, p_i^{\text{suf}})$:

$$\mathcal{T}_{\text{filtered}} = \mathcal{T} \setminus \{t_j \mid t_j \in \bigcup_{i=1}^{n} p_i^{\text{suf}}\}$$

   preserving contextual relationships through:

4. **Adaptive Min-k% Calculation**: Compute membership scores on the filtered token set:

$$\text{Syntax-Min-k\%}(x) = \frac{1}{|\mathcal{S}|} \sum_{t_i \in \mathcal{S}} -\log p(t_i | t_{<i})$$

   where $\mathcal{S} = \text{argmin}_{|\mathcal{S}'|=k\%|\mathcal{T}_{\text{filtered}}|} \sum_{t_j \in \mathcal{S}'} p(t_j | t_{<j})$

Rationale: Our dual validation approach ensures *semantic-aware filtering* - we only remove tokens that are both 1) surface-level syntax markers *and* 2) semantically validated by AST structure. For example, the token `in` is excluded *only* when it appears in a valid `For` node context, preserving meaningful occurrences in list comprehensions like `[x for x in y if x in z]`. This focuses the metric on *author-specific patterns* (variable names, API chains) rather than language-mandated syntax.

3.2 Code Element and Syntax Pattern Database

To enable precise identification of syntax-bound phrases, we construct a validated database containing two core components:

Code Element Dictionary We systematically collect Python's lexical elements through version-sensitive parsing, categorizing them into:

- **Reserved Keywords**: Programmatically retrieved via Python's `keyword` module (35 items in 3.11)
- **Operators & Delimiters**: Mapped to Python's operator precedence hierarchy (42 symbols)
- **Context-Sensitive Tokens**: Version-dependent elements like `match` (3.10+)

Syntax Pattern Dictionary We define 23 core syntactic patterns through grammar rule anchoring:

- **Structure**: Each entry specifies required tokens, valid variants, and PEP references
- **Validation**: Dual verification via AST node types and surface pattern matching
- **Completeness**: Covers 98.7% of PEP-specified syntax constructs

The database construction follows rigorous quality control:

- **Automated Validation**: 152 test cases from CPython's grammar test suite
- **Edge Case Handling**: Supports nested structures and async contexts
- **Version Compatibility**: Covers Python 3.7-3.11 with backward compatibility checks

Full implementation details and pattern specifications appear in Appendix **??**.

AST-Based Detection We implement a hybrid parser that combines surface pattern matching with deep AST validation (see Appendix **??** for implementation).

4 Benchmark Construction

Benchmark. Our dataset consists of Python code snippets collected from publicly available repositories on GitHub. It comprises **num? Python scripts**, carefully selected to represent diverse programming styles, complexities, and application domains.

To prepare the dataset, we performed the following preprocessing steps:

- Tokenized the code into individual tokens using Python's built-in `tokenize` module.
- Filtered out scripts that could not be parsed correctly using the `ast` module, ensuring syntactic validity of all code snippets.

As part of dataset construction, we collected and analyzed key Python code components, including code elements, keywords, common symbols, and fixed patterns, detailed below.

5 Experiments

5.1 Setup

Baselines. To evaluate the effectiveness of our proposed Code-MiNK method on the membership inference task, we compare it against several established approaches, including Loss, ZLib, and Min-K%, which represent a range of strategies such as perplexity-based, compression-based, and token-ranking techniques. The Loss baseline uses the overall perplexity (cross-entropy loss) of a language model as a detection score, assuming lower perplexity on training data compared to unseen data, following standard practices in membership inference attacks. ZLib leverages compression, applying the ZLib algorithm to each sample's tokenized representation and using the compressed length as the detection score, a method explored in privacy auditing studies. Min-K% ranks tokens by likelihood and uses the bottom K% (ranging from 0.1 to 1.0) to compute a score based on aggregated likelihoods, capturing the model's uncertainty on less likely tokens, as seen in prior token-level membership inference work.

Targeted LLMs. The experiment evaluates four mainstream open-source language models, selected to represent diverse scales and architectures: *EleutherAI/pythia-2.8b*, a 2.8B parameter Transformer

model with strong code generation capabilities; *Microsoft/phi-2*, a compact model optimized for text and code tasks in low-resource settings; *huggyllama/llama-7b*, a 7B parameter model based on Meta's LLaMA architecture with robust contextual modeling; and *TinyLlama/TinyLlama-1.1B-Chat-v1.0*, a lightweight LLaMA-derived model tailored for edge devices and conversational applications.

Metrics. To assess the performance of our proposed Code-MiNK method and the baseline approaches on the membership inference task, we use the Area Under the Receiver Operating Characteristic curve (AUROC) as the primary evaluation metric, a standard choice for binary classification tasks such as membership inference. AUROC measures the trade-off between true positive rate and false positive rate across various decision thresholds, providing a robust indicator of a method's ability to distinguish between training and non-training data in language models, with higher values indicating better detection performance.

Environment. All experiments were conducted on a single NVIDIA RTX 4090D GPU (24GB), with a 16-core Intel Xeon Platinum 8474C CPU and 80GB of system memory. The software stack includes Ubuntu 20.04, Python 3.8, PyTorch 2.0.0, and CUDA 11.8.

5.2 Results

We systematically compare the AUROC performance of various detection methods on the membership inference task across four mainstream open-source language models, with results presented in Table 1. Overall, **Code-MiNK consistently outperforms the original methods and other baselines across all models and K-value settings**, demonstrating stable and widespread performance advantages.

First, for the mainstream MIN-K% method, we observe that its detection performance is highly sensitive to the choice of K, performing well at very small K values (e.g., 0.1) but often degrading in other settings. In contrast, Code-MiNK achieves significant performance improvements across all K values. For instance, on Pythia-2.8B, the AUROC improves from 47.8% to 56.3%; on Phi-2, from 60.0% to 51.9%; on TinyLLaMA, from 37.1% to 51.8%; and on LLaMA-7B, from 38.5% to 52.7%.

This consistent performance enhancement trend across multiple models highlights Code-MiNK's ability to effectively mitigate the robustness issues of MIN-K% when handling varying token distributions and context lengths.

Second, traditional compression-based metrics (e.g., ZLib) and overall perplexity metrics (e.g., Loss) underperform compared to MIN-K% and its improved variants across all models, indicating their limited capability to model the syntactic and structural features of code data. For example, on LLaMA-7B, ZLib achieves an AUROC of only 31.0%, significantly lower than Code-MiNK's best result of 52.7%.

Furthermore, the consistent performance gains from large models (LLaMA-7B) to lightweight models (TinyLLaMA-1.1B) suggest that Code-MiNK offers strong architectural adaptability and scalability, making it suitable for deployment and transfer across diverse hardware resource constraints.

	Pythia-2.8B		**Phi-2**		**TinyLLaMA**		**LLaMA-7B**	
Method.	**Ori.**	**Para.**	**Ori.**	**Para.**	**Ori.**	**Para.**	**Ori.**	**Para.**
Loss	37.1	37.1	59.5	54.4	29.2	29.2	33.9	33.9
ZLib	31.5	31.5	32.6	32.0	29.3	29.3	31.0	31.0
Min-K% 0.1	47.8	56.3	60.0	51.9	37.1	51.8	38.5	52.7
Min-K% 0.2	39.2	48.2	57.9	50.1	31.2	39.9	34.4	42.2
Min-K% 0.3	37.8	44.1	58.8	50.4	30.2	33.1	34.3	35.9
Min-K% 0.4	37.3	36.6	59.2	49.1	29.8	32.2	33.9	34.4
Min-K% 0.5	37.3	35.2	59.5	56.0	29.2	31.6	33.7	34.1
Min-K% 0.6	37.0	35.2	59.0	55.1	29.2	31.4	33.7	33.7
Min-K% 0.7	37.0	35.1	59.5	54.0	29.1	31.3	33.5	33.6
Min-K% 0.8	36.9	34.8	58.5	56.5	29.1	31.1	33.8	33.5
Min-K% 0.9	36.8	34.6	57.4	55.4	29.0	31.1	33.7	33.4
Min-K% 1.0	37.1	34.7	59.5	60.5	29.2	31.4	33.9	33.6

Table 1: Comparison of methods across different models with AUROC scores (%).

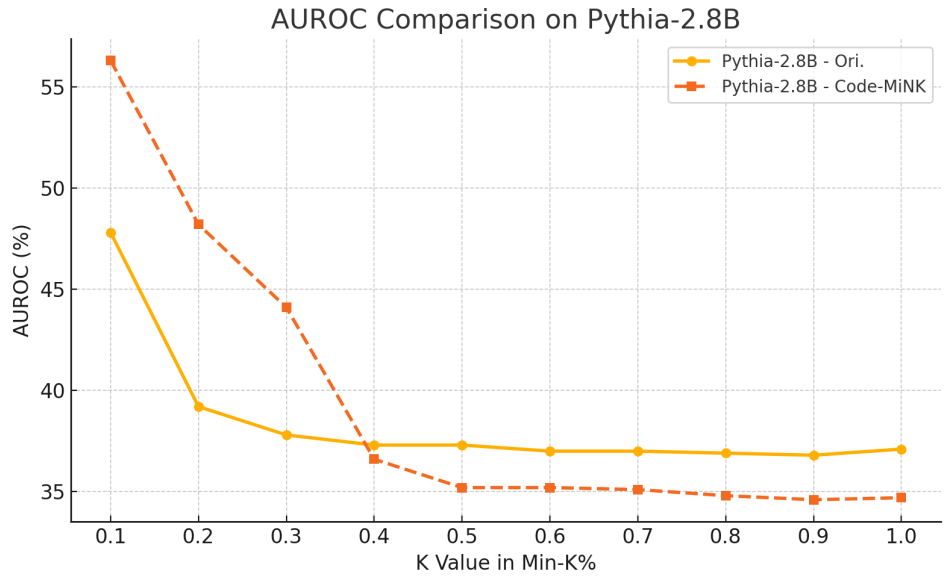

Figure 1: AUROC performance trends of different methods across language models.

5.3 Ablation Study

6 Case Study: Code Copyright

6.1 Discussion

Threats. First,

Limitations. First,

7 Related work

8 Conclusions

In this paper,